

Andamento do Musa CRM

Plano de **32 tarefas em 6 fases** para levar ao Musa os módulos operacionais que o BeautyPRO tem: agenda, precificação, financeiro, documentos e anamnese, estoque e fidelização. Quatro dias de trabalho, e **os seis módulos estão entregues**: agenda, precificação, financeiro, estoque, fidelização, documentos e anamnese. Restou uma tarefa das 32 — a comissão por profissional, opcional desde o plano original.

TAREFAS 31 concluídas · 1 na fila	ESFORÇO ENTREGUE ≈105 de ~115 dev-days planejados (91%)	MÓDULOS ENTREGUES 6 de 6 · todos no ar
RESTANTE 4 a 6 dev-days · só a comissão, que era opcional		

Fase 0 — Fundação

concluída · 21 dev-days reais

Estimada em 18,5 a 28 dev-days, custou 21. Cerca de quatro deles foram consumidos por dois incidentes de segurança que não estavam no plano. Não entrega tela nova — é o que faz as fases seguintes custarem o que foram estimadas.

T0.0	Fechar a API exposta helmet, CORS por origem, rate-limit e trava provisória de acesso	FEITO 26/08
------	---	-------------

T0.1	Repositório e higiene publicado em github.com/mulinotech/demo-musa , sem segredos no commit	FEITO 26/08
T0.2	Migrations e esquema versionado runner próprio, 6 migrations aplicadas, 11 índices que faltavam	FEITO 26/08
T0.3	Autenticação e autorização reais e-mail e senha com bcrypt, JWT de 12h, permissão verificada no servidor	FEITO 27/08
T0.4	Modularizar o app.js três fatias: código morto removido, camada de serviço extraída, 46 rotas em 14 arquivos	FEITO 27/08
T0.6	Harness de teste automatizado 21 casos com node:test e fetch nativo, sem dependência nova	FEITO 27/08
T0.5	Roteamento e code splitting no front CRM em rotas de verdade, tela de usuários, aviso de sessão expirada	FEITO 28/08

O que a T0.4 rendeu: o backend saiu de um arquivo de 2280 linhas para 20 arquivos com fronteira clara. O app.js hoje tem 4 linhas — só o ponto de entrada que o LiteSpeed executa. Foram 274 linhas de código morto removidas, a camada de serviço isolada em server/services/, e 46 rotas distribuídas em 14 arquivos, um por recurso.

Duas correções vieram de brinde: /api/users e /api/_migrate estavam presos dentro de um if (fs.existsSync(distPath)) — só existiam porque a pasta de build existe no servidor. E a comparação de prefixo da tabela de papéis usava index0f === 0, o que faria /api/logs-publicos herdar a regra de /api/logs. Há um teste travando esse comportamento agora.

O que a T0.5 rendeu, em um número: a página do site caiu de **1.005 kB para 331 kB** de JavaScript. O console inteiro estava sendo baixado por quem só queria ler a página de captação — inclusive o Recharts, que só serve ao painel de gráficos. Agora cada tela é um pacote próprio, carregado quando alguém a abre.

E o que rendeu em estrutura: `/crm/pacientes` virou endereço de verdade — dá para mandar um link, recarregar a página e usar o botão de voltar. A gestão de usuários saiu do terminal e virou tela. E o `CrmDashboard.tsx` deixou de ser onde as telas moram: agora é só o layout, e módulo novo entra como arquivo próprio mais uma linha de rota. Era esse o gargalo real das cinco fases seguintes.

Publicada e conferida em 28/08: 31 de 31 testes passando no servidor, e o smoke test das 16 rotas da API saiu *idêntico* à baseline de antes da modularização — nenhum comportamento de backend mudou no caminho. `/crm/pacientes` digitado direto responde 200, que é o que confirma que recarregar a página dentro do CRM não dá mais 404.

Uma trava que veio junto: tornar a gestão de usuários clicável criou dois cliques capazes de deixar o sistema sem nenhum administrador. O servidor agora recusa inativar ou rebaixar o último administrador ativo, e ninguém remove o próprio acesso. São dez testes novos — a suíte foi de 21 para 31.

O que os 31 testes cobrem — e o que não cobrem: eles travam autenticação e autorização, que é a camada onde um erro vira vazamento. Um deles verifica que o 401 não manda `WWW-Authenticate`, o cabeçalho que fazia o navegador sequestrar a tela de login. Cobrem também as travas da gestão de usuários. Não cobrem regra de negócio de módulo, porque ainda não existe nenhuma — o harness existe justamente para os testes da agenda, do estoque e da precificação terem onde nascer.

Há ainda uma verificação de navegação que roda um navegador de verdade sobre o front compilado: 30 conferências que teste de unidade não pega — se o site deixou mesmo de baixar o console, se cada aba abre sem erro no console do navegador, se quem não tem token cai no login, se o vendedor não vê o que não deve.

Fase 2 — Precificação e financeiro

entregue · 17 dev-days

O primeiro módulo, escolhido para vir antes da agenda por não depender de nada. Entregue inteiro, menos a comissão por profissional, que era opcional desde o plano.

aluguel, energia, contador e pró-labore viram um número: o custo por hora

T2.2	Motor de cálculo e o teste de referência função pura, 17 casos, o caso conferido à mão batendo centavo a centavo	FEITO 28/08
T2.3	Tela da calculadora, aplicar ao catálogo e histórico recalcula ao vivo, compara com o preço praticado e grava a decisão	FEITO 28/08
T2.4	Razão único, categorias, despesas e recorrência receita e despesa na mesma tabela; competência e caixa em colunas separadas	FEITO 28/08
T2.6	Resultado, fluxo de caixa e contas a pagar quatro números com variação contra o período anterior, e o alternador de base	FEITO 28/08
T2.5	Receita nascendo do atendimento fechou com a T1.4: concluir um atendimento lança a receita, a receber	FEITO 31/08
T2.7	Comissão por profissional e fechamento por período era opcional no plano desde o início	4-6 D

A conta que quase todo sistema erra. Margem, comissão, taxa de cartão e imposto incidem sobre o *preço de venda*, não sobre o custo. Com R\$ 120 de custo direto e 49,5% de percentuais, o preço certo é R\$ 237,62 — $\text{custo} \div (1 - \text{percentuais})$. Multiplicar o custo por 1,30 daria R\$ 156,00 e uma margem real de 19%, não de 30%: a clínica trabalharia de graça achando que está lucrando. Existe um teste que quebra se alguém "simplificar" essa conta.

O número que vende a tela. Na Limpeza de Pele Premium, cadastrada a R\$ 180, a conta pede R\$ 237,62. A tela escreve isso em português: *you are leaving R\$ 57,62 per service on the table — in 20 services per month, it's R\$ 1.152,40*. É o mesmo raciocínio que o concorrente usa como carro-chefe.

Uma armadilha encontrada no caminho. A duração no catálogo é texto livre: convive "60" com "40 a 60 minutos". Precificar pegando o primeiro número que aparece erraria justamente o valor por hora. A migration criou uma coluna de

minutos e só preencheu o que era inequívoco; para o resto, a tela pede a duração em vez de chutar.

O financeiro tem uma armadilha própria, e ela é maior. Competência e caixa dão resultados diferentes para o mesmo mês, e os dois estão certos: competência conta o que aconteceu, caixa conta o dinheiro que andou. No cenário de conferência do módulo, março fecha em **R\$ 400,00 por competência e R\$ 700,00 por caixa** — a diferença são R\$ 800 de aluguel lançado e ainda não pago. Um relatório que mostra um número sem dizer qual dos dois critérios usou é a causa nº 1 de alguém ver dois valores que não batem e parar de confiar no sistema inteiro. Por isso o alternador fica no topo e a base aparece escrita embaixo de cada valor.

Duas decisões estão embutidas no formato das tabelas, para não serem redecididas depois: receita e despesa vivem no mesmo razão, separadas por um campo de tipo — duas tabelas produzem, cedo ou tarde, dois "lucros" que não batem. E o valor é sempre positivo: o sinal vem do tipo. Guardar despesa como número negativo parece prático e vira erro de soma na primeira vez que alguém esquece o módulo.

A receita passou a nascer sozinha em 31/08. Concluir um atendimento na agenda lança a receita no financeiro, e a importação das sessões antigas virou o que sempre foi: um preenchimento de histórico. A aposta da T2.5 deu certo — o formato de origem escolhido na véspera era o mesmo que a agenda usaria, e nada precisou ser jogado fora. *Quase*: a chave apontava para a sessão clínica, e o evento aponta para o compromisso. Duas chaves para o mesmo fato fariam o sistema faturar de novo cada atendimento migrado; uma migration reapontou as duas para a mesma coisa antes que isso acontecesse.

Dois erros no mesmo dia, os dois registrados. O primeiro foi de produto: a rota que gera despesas recorrentes criava, num clique, todas as ocorrências desde a data de início — e uma recorrência cadastrada com data retroativa fez o painel acusar **R\$ 32.800 em contas vencidas que nunca existiram**. Não quebrou nada; só mentiu, que é pior. O padrão passou a ser apenas o mês corrente, e preencher o passado virou um ato deliberado. Virou função pura com três testes que descrevem o incidente.

O segundo foi operacional: a API limita 120 requisições por minuto, e um script de carga estourou o teto no meio do caminho. Metade dos lançamentos gravou, metade não, e a conferência final também levou 429 e devolveu zeros — parecia

que nada tinha funcionado. Ficou a regra: todo script que escreve em lote precisa de freio e de idempotência por conteúdo, para poder ser retomado.

A demo agora tem números. Cinco meses de operação semeados: uma clínica que sai de 19,5% e chega a 39,5% de margem, com receita de R\$ 21 mil a R\$ 30 mil por mês e duas contas em aberto para a faixa de contas a pagar ter o que mostrar.

Fase 1 — Agenda

entregue · ~20 dev-days

O módulo maior e o mais visível numa demonstração, estimado em 19 a 28 dias e entregue em cerca de 20. Calendário em três visões, regra de conflito, remarcação com histórico, o evento em que três outros módulos vão pendurar efeito, e o lembrete automático por WhatsApp com confirmação pela resposta da paciente.

T1.1	Modelo de agenda e regra de conflito tabela própria, grade semanal por profissional, 6 compromissos migrados do histórico	FEITO 30/08
T1.2	Rotas de agendamento criar, editar, remarcar, mudar status e consultar horários livres	FEITO 30/08
T1.3	Calendário dia, semana e mês clicar num espaço vazio já agenda naquele horário; conflito explicado na tela	FEITO 30/08
T1.4	O evento "atendimento realizado" serviço único, transacional e idempotente — fecha a T2.5 e prepara estoque e fidelização	FEITO 31/08
T1.5	Lembrete e confirmação por WhatsApp prévia antes do interruptor; nasce desligado; "1" confirma, "2" sinaliza remarcação	FEITO 31/08

A regra de conflito e o erro que passa em revisão. Dois compromissos conflitam quando um começa antes de o outro terminar e termina depois de o

outro começar. Escrita com "menor ou igual" em vez de "menor", a expressão continua parecendo certa — e transforma o dia inteiro de uma clínica em erro, porque uma agenda real é feita de horários encostados: 9h às 10h, 10h às 11h. Há um teste chamado *compromissos encostados não conflitam* exatamente para isso.

Remarcar não é editar a data. O sistema cria um compromisso novo apontando para o antigo e cancela o antigo. Editar a data no lugar apagaria para sempre a informação de quem remarca demais — que é justamente o número que a clínica quer.

A agenda não é a tabela de sessões clínicas. Usar a que já existia era o caminho curto, e encheria o prontuário de sessões fantasma: bloqueio de almoço, avaliação, retorno e encaixe ocupam a agenda sem serem tratamento. Os compromissos vindos do histórico entraram com hora 09:00 e uma observação dizendo que o horário é estimado — sem essa marca, alguém liga para a paciente confirmando um horário inventado.

Por que a T1.4 é o núcleo do projeto. Três módulos precisam reagir ao mesmo fato: financeiro lança a receita, estoque baixa o insumo, fidelização credita o ponto. Se cada um pendurar o próprio gatilho na tela de status, um dia alguém conclui o atendimento por outro caminho e um dos três não acontece — e ninguém descobre até o saldo não fechar. Existe **um** serviço que trata a conclusão, e ele é transacional: se o estoque não tiver saldo, o atendimento inteiro volta atrás. Estoque e fidelização já têm o encaixe pronto, vazio, esperando as Fases 3 e 5.

Concluir não é receber. A receita nasce com a data do fato e sem data de pagamento: vai para "a receber", e alguém marca quando o dinheiro entra. Carimbar pago na conclusão encheria o caixa de dinheiro que ninguém viu — a mesma armadilha de competência e caixa que o financeiro já tinha resolvido.

Desfazer existe porque erro de recepção existe. Marcar como realizado por engano acontece toda semana. Desfazer estorna com um lançamento novo, nunca apagando: o erro e a correção ficam lado a lado, e é isso que permite auditar uma divergência de saldo depois. A rota de exclusão recusa apagar lançamento vindo de atendimento, para ninguém "consertar" sumindo com a evidência. Só admin e gerente, com motivo obrigatório.

Dois defeitos apareceram na primeira saída real e foram corrigidos no mesmo dia. O horário voltava como 2026-04-14T12:00:00.000Z — um compromisso das 09:00 convertido para fuso universal. Duas consequências

silenciosas: o horário exibido passaria a depender do fuso de quem abre a tela, e agrupar por dia jogaria o compromisso da noite para o dia seguinte. E o título saía `SESSAO_TRATAMENTO0`: o campo parecia certo pelo nome, mas guarda um código, e a recepção veria seis blocos idênticos em maiúsculas.

A verificação hoje são 144 testes de servidor e 85 conferências de tela, estas últimas num navegador de verdade sobre o front compilado. As mais novas concluem um atendimento pela interface e checam que a tela *diz* o que aconteceu com o dinheiro — efeito colateral silencioso em valor é como se perde a confiança da pessoa no sistema.

O lembrete nasce desligado, e isso é a decisão de projeto da T1.5. O banco de demonstração tem telefone de gente real cadastrado como paciente. Um worker que sobe ligado dispara WhatsApp para essas pessoas na primeira vez que alguém abre o sistema — e mensagem enviada não tem desfazer. Na tela, a prévia vem *antes* do interruptor: mostra o texto integral que sairia, para quem sairia, e também quem *não* receberia e por quê ("paciente sem telefone", "status cancelado"). Ligar é um ato deliberado.

O gatilho é um instante, não uma faixa de horas. O desenho original mandava buscar compromissos entre 23 e 25 horas de antecedência. Nesta hospedagem o servidor recicla o processo quando ninguém acessa — e com faixa, tudo que vence enquanto o processo dorme nunca recebe lembrete, sem nenhum sinal de que faltou. A regra virou "já passou do momento e ainda não envie!": um despertar tardio manda atrasado, e a tela mostra o atraso como sintoma de que o disparo automático parou de rodar.

A resposta é comparada inteira, nunca "contém". 1 confirma o horário. 1 hora antes dá? não confirma nada — é uma pergunta, e vai para atendimento humano. Procurar o dígito dentro do texto marcaria essa paciente como confirmada e a clínica seguraria um horário que ninguém garantiu. E 2 sinaliza pedido de remarcação sem remarcar: trocar um horário incerto por outro inventado é pior do que não mexer.

Uma dependência foi dispensada. O plano autorizava `node-cron`. Para "a cada 15 minutos" a expressão `cron` não compra nada, e instalar pacote naquele servidor hoje significa reinstalar as dependências sob Node 22 — pendência ainda aberta. O relógio interno vive no próprio processo, e como ele morre quando o servidor recicla a aplicação, existe também uma rota de disparo para um `cron` do sistema chamar. As duas portas são idempotentes: rodar duas vezes não repete mensagem.

Estimada em 12 a 16 dias e entregue em cerca de 14. Produtos, lote, validade, custo médio, alertas, extrato legível, ficha técnica por serviço — e a baixa automática encaixada no evento que a Fase 1 deixou pronto e vazio.

T3.1	Modelo com lote e validade desde o primeiro dia saldo derivado do lote; nenhuma coluna de estoque no produto	FEITO 31/08
T3.2	FEFO, custo médio ponderado e alertas regras puras, 30 testes; lote vencido nunca sai automaticamente	FEITO 31/08
T3.3	Telas de saldo, entrada, extrato e ficha técnica alertas no topo; entrada mostra o custo médio antes de confirmar	FEITO 31/08
T3.4	Baixa no atendimento e custo variável da precificação saldo insuficiente derruba a conclusão inteira, nomeando o produto	FEITO 31/08

Um estoque que só conta unidades não serve para esta clínica. Toxina botulínica, ácido hialurônico e anestésico têm lote e validade, e rastrear lote é requisito sanitário — se um lote for recolhido, a clínica precisa saber em quem foi aplicado. Por isso lote existe desde a primeira migration, e não como "fase 2 do estoque": deixar para depois significaria migrar histórico sem lote.

O saldo não existe como campo. É a soma dos lotes, recalculada em cada consulta. Guardar o total também no produto custaria menos SQL e criaria duas fontes para o mesmo número — que divergem, sempre; a pergunta é só quando. Aí ninguém sabe qual está certa e o inventário inteiro perde a credibilidade.

E são dois saldos na tela, não um. Total é o que está na prateleira; utilizável é o que pode ser aplicado em alguém. No estoque semeado da demonstração, o Skinbooster tem **2 unidades e saldo utilizável zero** — o lote venceu. Ele aparece ao mesmo tempo no alerta crítico e no de reposição, e o mínimo compara com o utilizável: contar o vencido faria o sistema dizer "tem estoque" para algo que não pode ser usado em ninguém.

A saída é por validade, não por ordem de chegada. FEFO — *first expiry, first out*. Consumir o lote mais antigo, que é o reflexo automático, deixaria vencer na prateleira o lote que vence antes. Uma saída pode atravessar mais de um lote, e nesse caso gera um movimento por lote: registrar um só, com a soma, apagaria de qual lote saiu o que foi aplicado na paciente.

Saldo insuficiente derruba o atendimento inteiro. Decisão de produto, não de código. Concluir e deixar o estoque negativo parece mais gentil e é pior em todos os desdobramentos: a clínica descobre a falta no meio do próximo procedimento, o custo do atendimento fica errado e o inventário vira um número em que ninguém confia. A mensagem nomeia o produto e diz quanto falta — "erro ao concluir" manda chamar suporte; nomear faz a pessoa resolver na frente da tela.

Custo médio ponderado, nunca o último preço pago. Uma compra de emergência de 1 unidade a R\$ 300 faria as 20 antigas valerem R\$ 300 cada, e o preço sugerido subiria em cima de um custo que a clínica não tem. A tela mostra o custo antes e depois *antes* de confirmar a entrada, porque esse número vai direto para a precificação — mudar isso em silêncio significa que um dia o preço muda e ninguém sabe por quê.

A precificação passou a admitir a origem do custo de insumo. "Calculado pela ficha técnica (6 insumos)" ou "informado manualmente". R\$ 120 pode ser a soma real dos produtos com o custo de hoje ou um número digitado há um ano; os dois aparecem igual na conta e levam a decisões diferentes. Sem dizer qual é, o sistema faz um chute antigo parecer cálculo.

O que não foi migrado. As sessões antigas guardam insumos como texto livre — "1 seringa de AH". Não tem quantidade confiável, nem unidade, nem lote. Converter por adivinhação poluiria o saldo inicial com números que ninguém conseguiria auditar depois. O histórico fica onde está e o saldo começa por inventário. Na mesma linha, a ficha técnica foi semeada por palavra-chave do nome do serviço: **3 de 5 serviços casaram**, e os outros dois ficaram sem ficha de propósito — inventar o que eles consomem faria a baixa automática tirar o produto errado do estoque.

Um defeito encontrado na própria verificação. O harness de navegador guardava estado entre execuções: a segunda rodada encontrava os lembretes ligados e a ficha técnica salva pela primeira, e passava a afirmar coisas que não eram verdade num sistema recém-instalado. Verificação não-determinista é pior que verificação nenhuma, porque dá confiança falsa. Hoje ela zera o estado

antes de começar e passa duas vezes seguidas com o mesmo resultado: **170 testes de servidor e 119 conferências de tela.**

Fase 5 — Fidelização

entregue · ~12 dev-days

Estimada em 10 a 14 dias. Pontos por atendimento, catálogo de recompensas, resgate aplicado ao preço, expiração automática e o relatório que diz quanto o programa custa de verdade.

T5.1	Modelo, configurações e recompensas saldo é soma de transações; acúmulo retroativo dos atendimentos já concluídos	FEITO 31/08
T5.2	Acúmulo automático e extrato do paciente terceiro e último efeito a preencher o encaixe da T1.4	FEITO 31/08
T5.3	Resgate aplicado ao atendimento desconto entra antes da conclusão; o acúmulo cai sobre o valor pago	FEITO 31/08
T5.4	Expiração, relatório e telas expiração idempotente pelo id do acúmulo; custo real e passivo em circulação	FEITO 31/08

A regra que sustenta o programa inteiro é uma ordem. Desconto entra, atendimento é concluído, pontos acumulam — nessa sequência. Um resgate de R\$ 25 num atendimento de R\$ 250 acumula **225 pontos, não 250**. Se o acúmulo caísse sobre o valor cheio, a paciente geraria pontos com os próprios pontos e o programa se financiaria até quebrar, sem nenhum erro aparecer em tela. Por isso a rota *recusa* resgate em atendimento já concluído: depois da conclusão o crédito já foi feito sobre o preço de tabela, e a conta ficaria errada em favor da paciente, em silêncio.

Piso, nunca arredondamento. R\$ 249,90 com um ponto por real gera 249 pontos. Arredondar daria 250 — e creditar ponto que não foi conquistado é dívida silenciosa que cresce sozinha a cada atendimento.

A tela de configuração mostra a consequência antes de salvar. "Um atendimento de R\$ 250 gera 250 pontos, que valem R\$ 25,00 — 10% de volta." Trocar o valor do ponto para R\$ 0,50 faz a frase dizer *50% de volta*, com um aviso para comparar com a margem dos procedimentos. Nenhuma validação pegaria isso: o número não é inválido, é incompatível com a margem da clínica — e quem sabe disso é a pessoa, não o sistema.

Uma inconsistência do próprio plano, corrigida. O desenho original dizia que os pontos de estorno seriam sempre positivos. Não fecha: o estorno de um acúmulo indevido tem de ser negativo, e o estorno de um resgate cancelado tem de ser positivo — o mesmo tipo de lançamento, dois sinais opostos. O sinal passou a viver na própria linha, e o saldo é a soma simples. A regra escrita no código é: *o sinal segue o fato, nunca o tipo*.

Uma decisão que o contexto deixava em aberto, escolhida e documentada. Para saber quais pontos expiram, havia dois caminhos: amarrar cada resgate aos acúmulos de onde saiu, ou recalculando a fila a partir do extrato. Escolhemos o segundo — os acúmulos formam uma fila por data de validade, e todo consumo é aplicado contra ela em memória. Com tabela de ligação, cada resgate gravaria várias linhas e todo estorno teria de desfazer exatamente aquelas; é mais uma estrutura para ficar inconsistente. O preço da escolha, dito na cara: o extrato não responde "esses 50 pontos saíram do acúmulo de março ou de abril" — responde "o saldo é X e Y vence em tal data", que é o que a recepção e a paciente perguntam.

Dois defeitos meus, encontrados antes de rodar. O "desfazer resgate" devolvia os pontos em dobro — inseria o estorno e apagava o resgate — e recalculava o desconto a partir de um preço que já tinha sido alterado, o que devolveria o valor errado num desconto percentual. A correção foi gravar quantos reais o resgate abateu, e ela rendeu um número melhor de brinde: o relatório passou a mostrar o **custo real do programa** em vez da estimativa. Num procedimento de R\$ 1.800, um desconto de 20% custa R\$ 360 por 900 pontos — a tabela diria R\$ 90.

Dois números que a clínica precisa ver juntos. O custo do período é o dinheiro que os descontos tiraram. O *passivo em circulação* é quanto valeriam todos os saldos existentes se fossem resgatados hoje — na demonstração, R\$ 275. Não é dívida contábil, não é obrigação em dinheiro, e por isso não vira lançamento no financeiro: desconto não é dinheiro que saiu, é dinheiro que não

entrou, e a receita já foi lançada menor. Mas é o tamanho da promessa que a clínica já fez.

E um alerta que é oportunidade. Ponto que vira pó sem a paciente saber é promessa quebrada em silêncio. A ficha mostra quanto vence nos próximos 30 dias justamente para virar motivo de ligação — é esse o gancho de reengajamento que justifica um programa de pontos existir. Sem a equipe dizer o saldo no fim do atendimento, o recurso vira tabela vazia.

A barra de navegação quebrou, e o teste achou antes de você. Com um módulo novo por fase, ela chegou a treze abas, passou a ocupar três linhas e uma delas caiu fora da área visível — impossível de clicar. O remendo do dia foi agrupar em dois níveis e esconder a gestão atrás de um botão; ele durou algumas horas e virou a barra lateral descrita mais abaixo. O corte por **frequência de uso** sobreviveu: manter "Logs do Sistema" no mesmo nível da agenda do dia trata as duas como igualmente urgentes.

A verificação hoje são 197 testes de servidor e 157 conferências de tela. Entre elas, a que concluiu um atendimento, viu a receita nascer, o insumo baixar por validade, os pontos serem creditados — e depois desfez tudo, com os saldos voltando ao valor exato de antes.

Fase 4 — Documentos e anamnese

entregue · ~19 dev-days

Estimada em 16 a 23 dias. O Musa já *coletava* informação clínica — anamnese em texto livre e laudo de IA. O que faltava era forma jurídica: pergunta estruturada, termo de consentimento, conteúdo congelado, assinatura e trilha de acesso.

T4.1	Modelo e migração da anamnese existente 18 perguntas semeadas, 8 marcadas como contraindicação; coluna antiga preservada	FEITO 31/08
T4.2	Formulário dinâmico e imutabilidade perguntas condicionais; depois de gerado o conteúdo não muda	FEITO 31/08
T4.3	Alertas clínicos na ficha e na agenda	FEITO 31/08

faixa vermelha no compromisso, onde a profissional está antes de aplicar

T4.4

Assinatura eletrônica simples, hash e trilha

FEITO 31/08

canvas com dedo, SHA-256 do conteúdo, IP e data; sem PDF no servidor

T4.5

Guardas de LGPD

FEITO 31/08

vendedor bloqueado, leitura registrada, exportação do paciente, docs/LGPD.md

Um furo de segurança apareceu antes do módulo, no planejamento das permissões. A tabela de papéis casa por prefixo, e `/api/clients/:id/documents` cairia sob `/api/clients` — que não tem regra de leitura. A rota ficaria **aberta a qualquer autenticado, inclusive ao perfil comercial**: histórico de saúde de paciente visível para quem vende, sem nenhum erro aparecer em tela. A tabela passou a aceitar expressão regular além de prefixo, e a regra que fica escrita é simples: rota com id no meio do caminho precisa de padrão, não de prefixo.

Depois de gerado, o documento é imutável — e é isso que dá valor à assinatura. Corrigir significa emitir documento novo. O botão avisa antes de a pessoa clicar, não depois de ela tentar corrigir.

O código de integridade é do texto renderizado, não das respostas. Se fosse das respostas, o mesmo código valeria para dois textos diferentes — bastaria mudar o corpo do modelo. É o texto que a paciente *leu* que precisa ser provável. Na abertura o código é reconferido, e a página avisa se o conteúdo não corresponde: sinal de alteração feita por fora do sistema.

Editar um modelo cria a versão seguinte. Documento assinado aponta para a versão que foi assinada; alterar o modelo no lugar faria o histórico exibir perguntas que aquela paciente nunca viu, e a assinatura dela cobriria um texto que não existia.

Sem anamnese, a tela diz "ninguém avaliou" — nunca "nenhum alerta". São informações diferentes, e mostrar tranquilidade onde não houve verificação é mentira por omissão. Três estados distintos: contraindicação relatada, anamnese respondida sem contraindicação, e ninguém avaliou.

A faixa vermelha aparece no compromisso da agenda, não só na ficha do paciente — porque é ali que a profissional está segundos antes de aplicar o produto. Anamnese guardada num arquivo que ninguém abre não impede aplicação de toxina em gestante.

O PDF ficou fora do servidor, por decisão sua. O valor jurídico não vem do arquivo: vem do HTML congelado, do código de integridade e da trilha de IP e data — que existem de qualquer forma. Instalar o Chromium para gerar PDF custaria 170 MB e memória por documento numa hospedagem compartilhada, e o documento assinado abre numa página pronta para imprimir. Se um dia o arquivo no servidor virar necessário, entra sem refazer o módulo.

Sobre a assinatura, o texto na tela é deliberado: eletrônica *simples*, com validade entre as partes pela MP 2.200-2/2001 e pela Lei 14.063/2020 porque há registro de autoria e integridade — e **não** assinatura digital qualificada com certificado ICP-Brasil. Descrever o recurso como "assinatura digital certificada" seria afirmação falsa sobre o valor probatório do documento.

Dois defeitos foram encontrados pela verificação, não pela clínica. O primeiro: "gerar para assinatura" validava as respostas *gravadas*, não as da tela — a pessoa preenchia a anamnese inteira, clicava, e o sistema reclamava das perguntas que ela acabou de responder. O segundo era pior porque era silencioso: o campo de assinatura descartava um traço único. A função que finaliza o traço lia o valor antigo do estado, então a paciente assinava, levantava o dedo e o botão continuava bloqueado — só funcionava no segundo traço.

O que o sistema entrega de LGPD, e o que não entrega. Entrega: acesso por papel testado, trilha de toda leitura de documento assinado, nenhum arquivo em diretório público, verificação de integridade e exportação dos dados do paciente. Não entrega, porque não é decisão de software: prazo de retenção, base legal de cada finalidade e política de exclusão — que dependem da clínica com o jurídico dela. O docs/LGPD.md lista tabela por tabela o que é dado sensível, e registra as dívidas técnicas com efeito em privacidade que continuam abertas.

O que falta

4 a 6 dev-days

Do plano original, restou uma tarefa — e ela já era marcada como opcional. O que existe de trabalho pendente não é módulo: é infraestrutura e calibração,

listado no fim desta página.

1	Financeiro — entregue resta apenas a comissão por profissional, opcional desde o plano	1 tarefa	4–6 d
3	Estoque — entregue resta preencher a ficha técnica dos dois serviços que não casaram por palavra-chave	—	meio dia
4	Documentos e anamnese — entregue resta a clínica definir prazo de retenção e revisar o texto do termo com o jurídico	—	jurídico
5	Fidelização — entregue combinar com a equipe que o saldo será dito ao paciente no fim de cada atendimento	—	conversa

Fora do plano

Boa parte do vaivém destes dois dias foi trabalho que não estava no backlog — e que não era opcional.

Duas exposições de dados fechadas

A API não exigia autenticação: uma chamada sem credencial devolvia a ficha dos pacientes com anamnese e foto. E quatro telas comparavam senha no próprio JavaScript, com as strings publicadas no bundle e no repositório.

Trilha de auditoria confiável

Nove rotas gravavam o autor da ação como o que o cliente dizia ser. Um log que aceita o autor por cabeçalho não audita nada. Passou a vir do token.

Smoke test versionado

Escopo do vendedor restaurado

O filtro que limitava o vendedor a ver só os próprios pacientes dependia de um cabeçalho enviado pelo navegador. Ao trocar para JWT ele parou de funcionar em silêncio; agora vem do token.

Gestão de usuários

Criar acesso novo não envolve mais editar arquivo nem reiniciar servidor. Os três sócios têm credencial própria; nenhuma senha é compartilhada.

Pacote de contexto reescrito

Dezesseis rotas verificadas em um comando, com linha de base guardada. Foi ele que encontrou a rota de logs quebrada antes de a refatoração começar, e que validou cada uma das três fatias da T0.4.

O AGENTS.md e o OPERACOES.md descreviam um sistema que deixou de existir. Refeitos, com as duas regras de segurança que este projeto já violou.

A navegação

fora do plano · 31/08

Com os seis módulos no ar, a barra de topo tinha catorze itens e metade deles vivia atrás de um botão suspenso. O layout estava resolvido; a descoberta, não. Quem não sabia que o botão existia não sabia que o estoque existia.

Na vertical o espaço é praticamente ilimitado — e essa é a única razão da mudança. A barra lateral mostra os catorze itens ao mesmo tempo, agrupados em "Dia a dia" e "Gestão", sem nada escondido. É a propriedade a preservar: se um dia a lista não couber, agrupe ou role, não volte a esconder.

No desktop ela recolhe para só os ícones — 240 px viram 72 px — e a escolha fica guardada no navegador. Num notebook de 13" isso é largura útil que ninguém quer reconquistar todo dia. **No celular e no tablet ela virou gaveta**, com o botão de menu no topo, fundo escurecido atrás, e fecha de três formas: no toque fora, no Esc e sozinha quando você troca de tela. As duas últimas não são luxo — o menu suspenso anterior prendia quem o abrisse sem querer, e gaveta aberta por cima da tela nova esconde justamente o que a pessoa acabou de pedir. A paleta é a mesma, item por item.

O mapa de telas saiu dos componentes para um arquivo só. A lateral desenha os itens, o cabeçalho lê dali o nome da tela aberta. Duas cópias divergiriam no primeiro módulo novo.

E foi consolidar essa lista que revelou um problema de verdade. O item da Fidelidade estava aberto a todos os papéis. A tela mostra custo do programa e passivo em circulação — dado da mesma natureza do preço, que o perfil comercial já não vê. Fechado nos dois lados, a tela e a rota do relatório, porque esconder o menu sem fechar a rota não é segurança, é decoração. *O saldo de um paciente* continua legível por todos: é a recepção que informa isso a ele no fim do atendimento.

A verificação são 222 testes de servidor e 196 conferências em navegador de verdade, incluindo a gaveta num aparelho de 390 × 844 e o recolhimento persistindo entre recargas.

Pendências abertas

Precisam de decisão ou de uma janela de trabalho

As credenciais de banco no `.env` do servidor estão defasadas — a aplicação usa as que o painel injeta — e por isso continua existindo uma rota temporária de migration, que só pode sair depois que o `.env` for corrigido. As outras aplicações Node no mesmo servidor nunca foram auditadas: se saíram do mesmo molde, têm o mesmo padrão de senha no front. O `node_modules` do servidor foi instalado sob Node 18 e por isso o build sai da máquina local. E as duas senhas que estavam no bundle continuam no histórico do git: estão queimadas e não podem ser reutilizadas em nenhum sistema da Mulino.

Próximo passo

nada de módulo

O plano de 32 tarefas fechou em 31. A que faltou — comissão por profissional — era marcada como opcional desde o início. O que resta é de outra natureza, e nada disso é módulo novo:

Um cron no servidor, meia hora de trabalho. Os dois workers — lembrete de WhatsApp e expiração de pontos — só rodam enquanto a aplicação está acordada, e a hospedagem recicla o processo quando ninguém acessa. As duas rotas de disparo existem e são idempotentes; falta um cron chamando cada uma, com um token de serviço.

A ficha técnica de dois serviços que não casaram com nenhuma palavra-chave na semente. Fica sem ficha de propósito — inventar o que um procedimento consome faria a baixa automática tirar o produto errado do estoque. São 15 minutos na aba de fichas técnicas.

A calibração do programa de pontos contra a margem real dos procedimentos. Hoje está em 10% de volta; a tela mostra a consequência ao vivo, e a decisão é de negócio.

E três conversas que não são técnicas: combinar com a equipe que o saldo de pontos é dito ao paciente no fim de cada atendimento (sem isso o programa vira tabela vazia); revisar o texto do termo de consentimento com o jurídico; e definir o prazo de retenção de prontuário, que o sistema registra mas não decide.

Estimativas em dev-days de um desenvolvedor sênior, cerca de 6 horas produtivas. Prazo de calendário a 21 dias úteis por mês e 75% de foco, com 15% de custo de coordenação na dupla. Os números vêm de leitura do código, não da velocidade histórica do time — calibre pelo primeiro módulo entregue.